Domain-Specific RAG Chatbot

Major Project of AI — Project Report

Student: Nikhil D P
Batch: Artificial Intelligence Program June (A) 2026 Batch

GitHub Repository: https://github.com/nikhilaaradhya3-cpu/domain-rag-chatbot
Live Demo: https://domain-rag-chatbot-3isqgaqdft9uqlbvmvm8ru.streamlit.app/

1. Project Objective

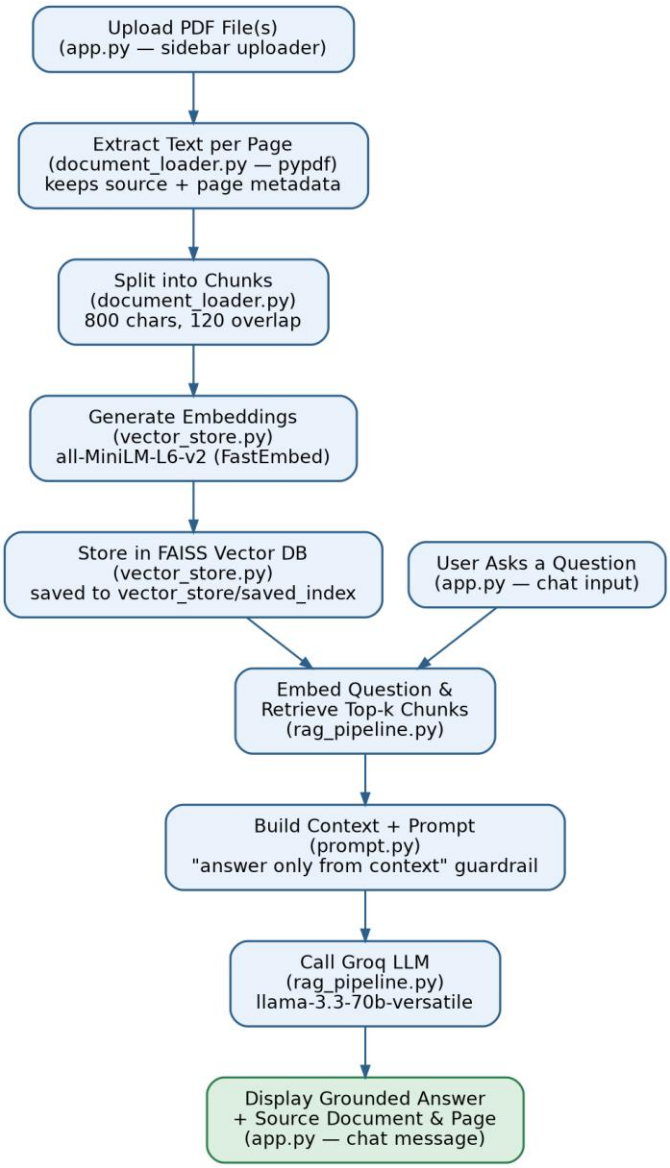
The goal of this project is to build a reliable question-answering chatbot that uses information from user-uploaded PDF documents. The chatbot retrieves the most relevant passages from the uploaded documents and generates an answer based only on those passages, citing the exact source document and page number, and explicitly declining to answer when the information is not present.

2. Problem Statement

Large documents such as policy manuals, course notes, and training material are difficult to search manually, often requiring a user to read many pages to find a single answer. This project solves that problem using Retrieval-Augmented Generation (RAG), allowing users to upload documents and ask questions in natural language.

3. System Architecture and RAG Workflow

The application follows the standard Retrieval-Augmented Generation pipeline shown below: documents are processed and indexed once, and every user question is answered by retrieving relevant chunks from that index before generating a response.



4. Tools and Technologies

Project Part	Tool / Technology
Programming language	Python
PDF text extraction	pypdf
Text splitting	LangChain RecursiveCharacterTextSplitter (800 chars, 120 overlap)
Embeddings	Sentence Transformers — all-MiniLM-L6-v2 (via FastEmbed)
Vector database	FAISS
Language model	Groq — Llama 3.3 70B Versatile
RAG framework	LangChain

User interface	Streamlit
Environment variables	python-dotenv
Deployment	Streamlit Community Cloud
Version control	Git and GitHub

5. Main Modules

Module 1: Document Upload
app.py provides a sidebar file uploader that accepts one or more PDF files, validates them, and lists the uploaded file names before processing.
Module 2: Text Extraction
document_loader.py reads every page of each PDF using pypdf, preserving the source file name and page number as metadata for later citation, and safely skips empty pages.
Module 3: Chunking
document_loader.py splits extracted text using LangChain's RecursiveCharacterTextSplitter with a chunk size of 800 characters and 120 characters of overlap, keeping connected ideas from being split apart.
Module 4: Embeddings and Vector Store
vector_store.py converts every chunk into a numerical embedding using the all-MiniLM-L6-v2 model via FastEmbed, then stores the embeddings and metadata in a FAISS vector index, which can be saved locally for reuse.
Module 5: Retrieval
rag_pipeline.py embeds the user's question and searches FAISS for the top-4 most similar chunks, retaining each chunk's source document and page number.
Module 6: Answer Generation
prompt.py defines a strict system prompt instructing the model to answer only from the supplied context and to state clearly when information is not available rather than inventing facts. rag_pipeline.py sends this prompt with the retrieved context and question to Groq's Llama 3.3 70B model.
Module 7: Streamlit Interface
app.py ties everything together: a sidebar PDF uploader and Process Documents button, a chat input box with persistent chat history, a source citation shown below every answer, and a Clear Chat button.

6. Testing and Evaluation

The chatbot was tested against 18 questions spanning two documents from different domains — an HR policy handbook (Company_HR_Policy.pdf) and a set of machine learning course notes (Machine_Learning_Course_Notes.pdf) — plus two questions with no answer in either document, used to verify refusal behavior. The full question set, along with the expected source, the chatbot's actual answer, the retrieved source, and a pass/fail result for each question, is recorded in tests/test_questions.csv in the repository.

Sample Question	Expected Source
What is the leave policy for casual leave?	Company_HR_Policy.pdf, p.3
What is overfitting in machine learning?	Machine_Learning_Course_Notes.pdf, p.4

Who is the CEO of the company?	Not available (refusal expected)

Testing focused on six criteria: retrieval accuracy (correct document and page found), answer correctness, groundedness (no unsupported claims), refusal quality when the answer is absent, source citation quality, and response time.

7. Responsible AI and Security

• API keys are stored in a local .env file and excluded from GitHub via .gitignore — never exposed in code or the repository.
• The chatbot answers only from the supplied document context and explicitly states when information is not available, rather than inventing facts.
• Users are shown the source document and page number for every answer so claims can be independently verified.
• Users are advised to verify high-stakes information rather than relying on the chatbot's answer alone.
• Uploaded file types are restricted to PDF.

8. Limitations and Future Work

The current version processes documents freshly within each session and does not support scanned/image-based PDFs without OCR, conversation memory for follow-up questions, or multi-user persistent document collections. Planned improvements include OCR support, conversation memory, a FastAPI backend for external clients, and Docker-based deployment.

9. Final Deliverables

• Working Streamlit application (deployed link above)
• Complete source code (GitHub repository)
• Sample PDF documents (sample_documents/)
• requirements.txt
• README with setup and usage instructions
• Architecture/workflow diagram (docs/architecture.png)
• Testing sheet with 18 questions (tests/test_questions.csv)
• GitHub repository
• Short project report and demonstration video